

Developing interactive web applications in R using Shiny

You can construct interactive web apps in R using the Shiny package. Apps can be hosted on a website, embedded in R Markdown documents, or used to develop dashboards and visualisations. CSS themes, HTML widgets, and JavaScript actions can all be used to customise your Shiny apps further.

Shiny is an R tool that makes it simple to create interactive web apps. It allows you to extend your R code to the web so that it may benefit from the larger community of people who can use it.

In addition to Shiny's built-in features, there are a number of third-party extension packages available, such as `shinythemes`, `shinydashboard`, and `shinyjs`.

There are various apps that can be developed using Shiny. Here is a list of a few of them.

- Machine learning based web apps
- Web apps with dynamic controls
- Data-driven dashboards
- Interactive apps for multiple data sets
- Real-time data visualisation panels
- Data collection forms

Shiny web apps can be divided into the following:

- User interface
- Server function
- Shiny app function

For better understanding, you can refer to the examples and use cases available at <https://shiny.rstudio.com/gallery/>.

One of the apps developed using Shiny is shown in Figure 2 (<https://shiny.rstudio.com/gallery/radiant.html>).

Generation of a sales dashboard

The code snippet for a web app related to a sales dashboard is given below. This dashboard has multiple controls and user interface modules to view the data.

First, the Shiny package is installed and then it is called in the code so that the output can be presented in the form of a web page.

```
library(shiny)
library(dplyr)

sales <- vroom::vroom("salesdata.csv", na = "")
ui <- fluidPage(
  titlePanel("Dashboard for Sales Data"),
  sidebarLayout(
    sidebarPanel(
      selectInput("territories", "territories", choices =
unique(sales$territories)),
      selectInput("Customers", "Customer", choices =
```

```
NULL),
      selectInput("orders", "Order number", choices = NULL,
size = 5, selectize = FALSE),
    ),
    mainPanel(
      uiOutput("customer"),
      tableOutput("data")
    )
  )
)

server <- function(input, output, session) {
  territories <- reactive({
    req(input$territories)
    filter(sales, territories == input$territories)
  })
  customer <- reactive({
    req(input$Customers)
    filter(territories(), Customers == input$Customers)
  })

  output$customer <- renderUI({
    row <- customer()[1, ]
    tags$div(
      class = "well",
      tags$p(tags$strong("Name: "), row$customers),
      tags$p(tags$strong("Phone: "), row$contact),
      tags$p(tags$strong("Contact: "), row$name, " ",
row$name)
    )
  })

  order <- reactive({
    req(input$order)
    customer() %>%
      filter(ORDER == input$order) %>%
      arrange(OLNUMBER) %>%
      select(pline, qty, price, sales, status)
  })

  output$data <- renderTable(order())

  observeEvent(territories(), {
    updateSelectInput(session, "Customers", choices = unique(
territories())$Customers), selected = character())
  })
  observeEvent(customer(), {
    updateSelectInput(session, "order", choices =
unique(customer())$order))
  })
}

shinyApp(ui, server)
```